Datawarehouse

Partisi

Oleh: Wiratmoko Yuwono
Tessy Badriyah
Rengga Asmara



Politeknik Elektronika Negeri Surabaya
Departemen Teknik Informatika dan Komputer

Tujuan Perkuliahan

- Mempelajari partisi data



Pengantar

- Datawarehouse seringkali berisi data yang besar dan membutuhkan teknik untuk mengatur tabel yang besar ini dan untuk mendapatkan performa query yang bagus pada beberapa gabungan tabel yang besar.
- Diskusi :

1. Tipe partisi
2. Partition-Wise Joins
3. Maintenance partisi
4. Partisi dan sub partisi kolom dan key
5. Batas Partisi untuk Range Partitioning
6. Index partisi

1. Tipe Partisi

- Range Partitioning
- Hash Partitioning
- List Partitioning
- Composite Partitioning

Range Partitioning

- Range partitioning memetakan data ke partisi berbasis rentang nilai pada sebuah key partisi.
- Contoh :

```
CREATE TABLE sales_range
(salesman_id NUMBER(5),
salesman_name VARCHAR2(30),
sales_amount NUMBER(10),
sales_date DATE)
COMPRESS
PARTITION BY RANGE(sales_date)
(PARTITION sales_jan2000 VALUES LESS THAN(TO_DATE('02/01/2000','DD/MM/YYYY')),
PARTITION sales_feb2000 VALUES LESS THAN(TO_DATE('03/01/2000','DD/MM/YYYY')),
PARTITION sales_mar2000 VALUES LESS THAN(TO_DATE('04/01/2000','DD/MM/YYYY')),
PARTITION sales_apr2000 VALUES LESS THAN(TO_DATE('05/01/2000','DD/MM/YYYY')));
```

Hash Partitioning

- Hash partitioning memetakan data ke partisi berbasis algoritma hashing dimana Oracle Mengaplikasikannya kedalam partisi key yang anda identifikasi.
- Algoritma hashing mendistribusikan baris secara rata kedalam partisi yang disediakan, dengan memberikan ukuran partisi yang sama.
- Hash partitioning adalah metode ideal untuk mendistribusikan data secara rata pada device yang berbeda, khusus bilamana data bukan bersifat historikal.

Hash Partitioning

- Contoh

```
CREATE TABLE sales_hash
(salesman_id NUMBER(5),
salesman_name VARCHAR2(30),
sales_amount NUMBER(10),
week_no NUMBER(2))
PARTITION BY HASH(salesman_id)
PARTITIONS 4;
```



List Partitioning

- List partitioning berguna bila mana anda dapat mengelompokan partisi data dalam urutan list
- Contoh :

```
CREATE TABLE sales_list
(salesman_id NUMBER(5),
salesman_name VARCHAR2(30),
sales_state VARCHAR2(20),
sales_amount NUMBER(10),
sales_date DATE)
PARTITION BY LIST(sales_state)
(PARTITION sales_west VALUES('California', 'Hawaii') COMPRESS,
PARTITION sales_east VALUES('New York', 'Virginia', 'Florida'),
PARTITION sales_central VALUES('Texas', 'Illinois'));
```

Composite Partitioning

- Anda dapat menggabung partisi data pada range, hash dan list partitioning, ini dinamakan composite partitioning.



2. Partition-Wise Joins

- Partition-wise joins mengurangi waktu query response time dengan meminimalkan pertukaran data pada eksekusi paralel servers dimana eksekusi join dilakukan secara pararel. Ini secara signifikan mengurangi waktu respon dan memperbaiki penggunaan resource CPU dan memory.
- Partition-wise joins ada 2 tipe yaitu full atau partial.

Full Partition-Wise Joins

- Full partition-wise join membagi large join kedalam smaller joins diantara sepasang partisi dari 2 buah join tabel. Untuk menggunakan feature ini, anda harus meng-equipartisi di kedua tabel pada join key.
- Sebagai contoh, pada large join diantara tabel sales dan tabel customer pada kolom customerid. Query " mencari semua records dari

  customer yang membeli lebih dari 100 articles pada Quarter 3 tahun 1999"



Full Partition-Wise Joins: Hash-Hash

- SQL-nya sbb:

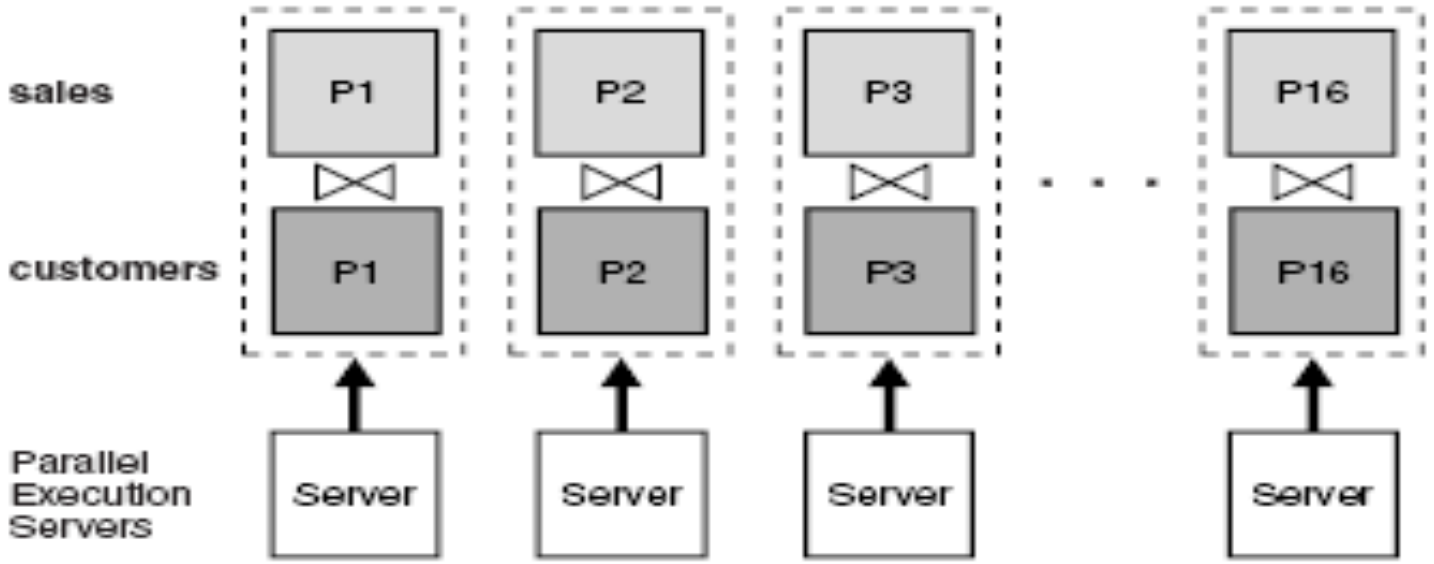
```
SELECT c.cust_last_name, COUNT(*)
FROM sales s, customers c
WHERE s.cust_id = c.cust_id AND
s.time_id BETWEEN TO_DATE('01-JUL-1999', 'DD-MON-YYYY') AND
(TO_DATE('01-OCT-1999', 'DD-MON-YYYY'))
GROUP BY c.cust_last_name HAVING COUNT(*) > 100;
```

Large join ini typical dalam data warehousing . Keseluruhan tabel customer dijoinkan dengan satu quarter pada sales data. Dalam large data warehouse, ini berarti Menjoinkan jutaan baris. Metode join yang digunakan pada kasus ini jelas hash join. Anda dapat mengurangi waktu proses dari hash join ini bilamana kedua buah tabel di equipartitioned pada kolom customerid. Enableskan full partition-wise join.

Full Partition-Wise Joins: Hash-Hash

- Ini adalah metode yang sederhana: tabel customers dan sales di partisi dengan hash kedalam 16 partisi, pada kolom s_customerid dan c_customerid.

Figure 5–1 Parallel Execution of a Full Partition-wise Join

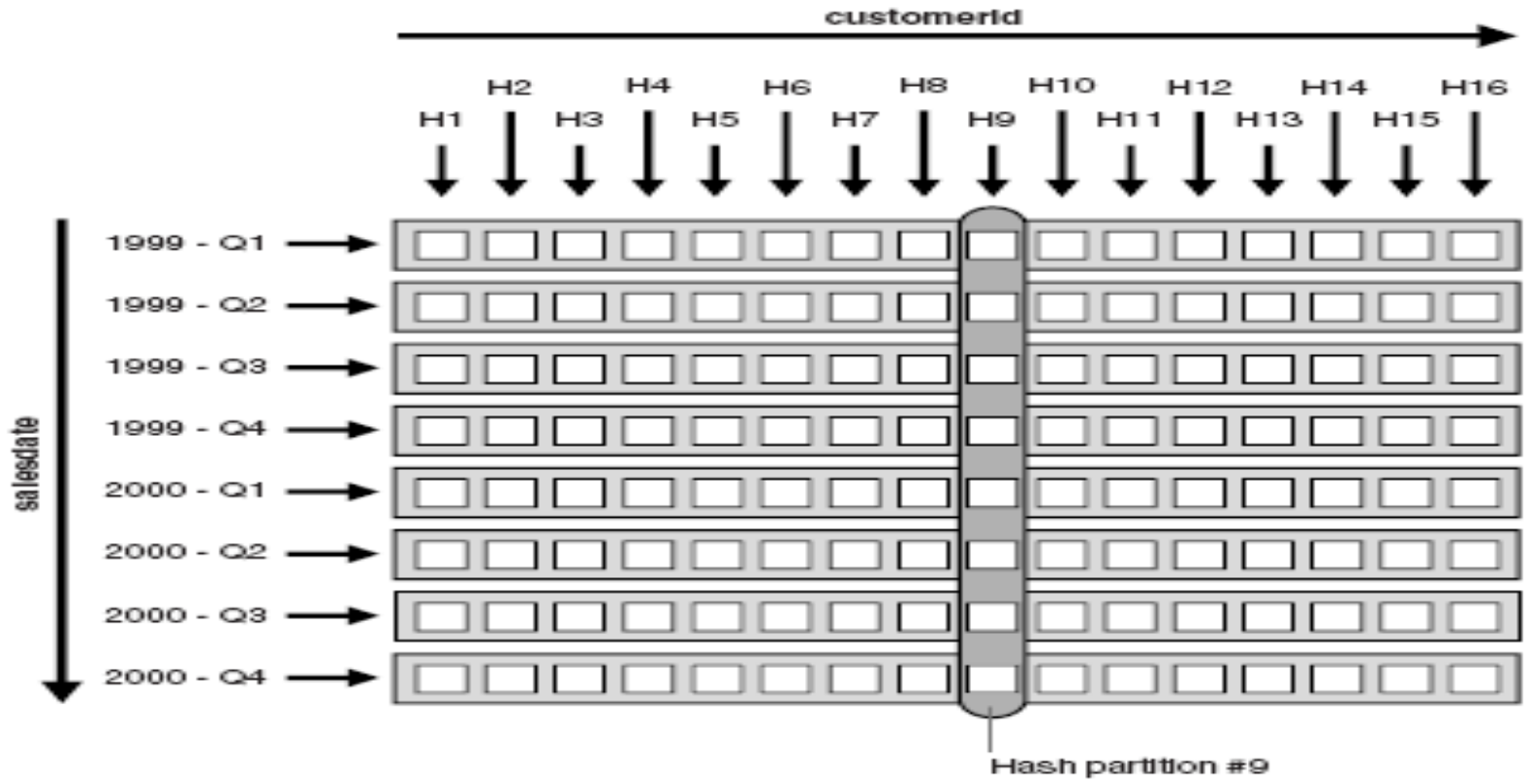


Full Partition-Wise Joins: (Composite-Hash)-Hash

- Metode ini adalah variasi dari metode hash-hash. Tabel sales adalah tipikal contoh dari tabel yang menyimpan data histori.
- Sebagai contoh, asumsikan anda ingin mempartisi tabel sales kedalam 8 partisi dengan range pada kolom s_salesdate. Juga asumsikan anda mempunyai 2 tahun data dan 1 partisi mempresentasikan 1 quarter (1 quarter=3 bulan).
- Partisi tabel sales dengan range pada s_salesdate
- Dan kemudian buat subpartisi pada tiap partisi dengan hash pada s_customerid using 16 subpartisi dari tiap partisi, total menghasilkan 128 subpartisi.

Full Partition-Wise Joins: (Composite-Hash)-Hash

Figure 5–2 Range and Hash Partitions of a Composite Table



(Composite-Hash)-Hash partitioning is effective because it lets you combine pruning (on `s_salesdate`) with a full partition-wise join (on `customerid`). In the previous example query, pruning is achieved by scanning only the subpartitions corresponding to Q3 of 1999, in other words, row number 3 in Figure 5–2. Oracle then joins these subpartitions with the customer table, using a full partition-wise join.

Full Partition-Wise Joins

- Anda bisa gunakan metode yang lain sesuai dengan kasusnya, spt :

- Full Partition-Wise Joins: (Composite-List)-List

- Full Partition-Wise Joins: (Composite-Composite (Hash/List Dimension)

- Full Partition-Wise Joins: Range-Range and List-List

- Full Partition-Wise Joins: Range-Composite, Composite-Composite (Range Dimension)

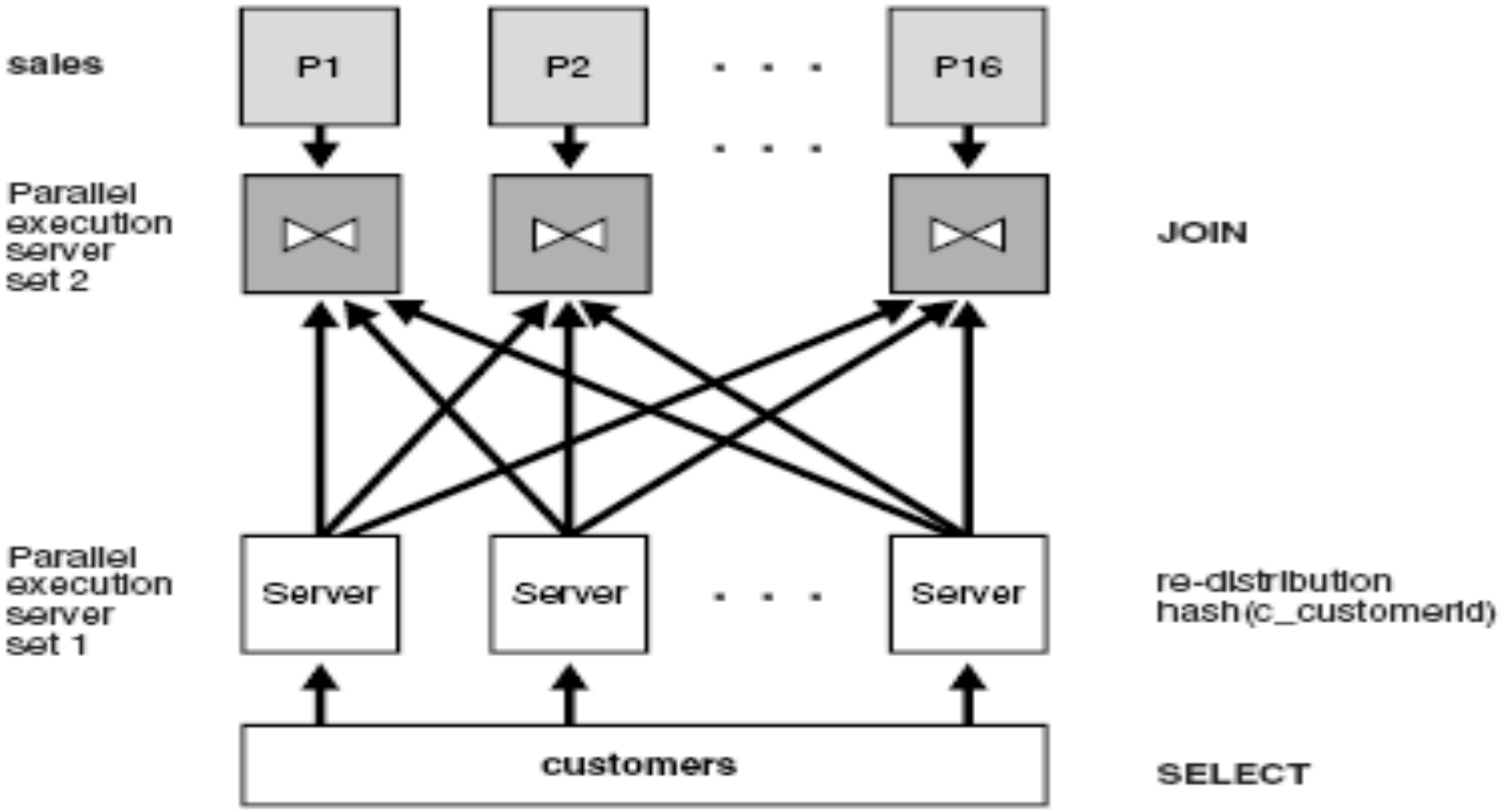


Partial Partition-Wise Joins

- Oracle dapat menggunakan partial partition-wise joins hanya pada paralel. Tidak seperti full partition-wise joins, partial partition-wise joins hanya mempartisi satu tabel dalam join key, tidak kedua-duanya.
- Untuk mengeksekusi partial partition-wise join, Oracle secara dinamis merepartisi tabel yang lain
- Berdasarkan referensi tabel yang dipartisi. Ketika tabel yang lain direpartisi, maka eksekusinya mirip dengan full partition-wise join.
- Pada kondisi join Primary dan Foreign Key, partisi hanya foreign key saja, karena biasanya tabel yang berisi foreign key lebih banyak transaksi didalamnya.

Partial Partition-Wise Joins: Hash-List

Figure 5–3 Partial Partition-Wise Join



Partial Partition-Wise Joins: Hash-List

- Contoh metode sederhana pada partial partition-wise join adalah mempartisi tabel sales dengan hash pada s_customerid. Jumlah partisi mencerminkan maksimum derajat pararel proses.
- Pada contoh jumlah partisi pada tabel sales berjumlah 16

Partial Partition-Wise Joins

- Anda bisa gunakan metode yang lain seperti

- **Partial Partition-Wise Joins: Composite**

  contoh : gunakan range partisi pada tabel sales di kolom s_salesdate, kemudian sub partisi dengan hash pada kolom s_customerid

- **Partial Partition-Wise Joins: Range**

Keuntungan Partition-Wise Joins

- Mengurangi Communications Overhead
- Mengurangi Memory Requirements

Maintenance Partisi

- Dibahas di bagian maintenance data warehouse

Partisi dan Sub Partisi Kolom dan Key

- Kolom partisi (atau kolom sub partisi) dari sebuah tabel atau index terdiri atas kolom yang diurutkan dimana nilainya mencerminkan bagaimana harus mempartisi atau mensubpartisi, kolom yang dipartisi bisa sampai 16 kolom dan tidak boleh mengandung kolom bertipe berikut ini :
  1. Level atau Rowid Psedocolumn
  2. Tipe data Rowid
  3. Nested Table, VARRAY, object type atau Ref Column
  4. LOB Column (BLOB, CLOB, NCLOB atau BFILE datatype)

Partisi dan Sub Partisi Kolom dan Key

- Row partitioning key adalah nilai yang telah diurutkan dari partisi kolom. Seperti dalam composite partitioning subpartitioning key adalah urutan list pada subpartitioning kolom.
- Oracle mengaplikasikan metode range, hash dan list pada tiap baris partitioning key atau sub partitioning key untuk mengambarkan bagaiman partisi atau sub partisi dilakukan.

Batas Partisi untuk Range Partitioning

- Pada partisi tipe range untuk tabel atau index, setiap kunci partisi pada setiap baris dibandingkan dengan batas atas dan batas bawah untuk mengambarkannya dijelaskan sebagai berikut :

1. Setiap partisi pada range partisi table atau index mempunyai noninclusive batas atas yang ditandai dengan klausa VALUES LESS THAN
2. Setiap partisi kecuali partisi pertama juga mempunyai inclusive batas bawah, dimana dispesifikasikan dengan VALUES LESS THAN pada partisi terbawah sebelumnya.

Batas Partisi untuk Range Partitioning

- Jika anda memasukkan baris kedalam tabel dan baris tersebut mempunyai kunci partisi yang lebih besar atau sama dengan batas partisi pada partisi yang terbesar, maka anda akan mengalami kegagalan memasukan data tersebut.

Batas Partisi untuk Range Partitioning

- MaxValue

  anda dapat menspesifikasikan kata kunci MaxValue untuk setiap nilai dalam batas partisi. Kunci ini merepresentasikan nilai virtual yang tak terbatas dimana disorting lebih tinggi daripada nilai yang lain untuk tipe data termasuk nilai NULL.

  Sebagai contoh, anda mempartisi tabel OFFICE pada kolom STATE (char(10)) kedalam 3 partisi dengan batasan partisi sebagai berikut :

- VALUES LESS THAN ('I'): States dimana dimulai dengan huruf A sampai dengan H
- VALUES LESS THAN (‘S'): States dimana dimulai dengan huruf I sampai dengan R
- VALUES LESS THAN (MAXVALUE): States dimana dimulai dengan huruf S sampai dengan Z ditambah spesial code untuk area NON-US

Batas Partisi untuk Range Partitioning

- NULL

  NULL tidak dapat dispesifikasikan pada nilai batas partisi, string yang kosong juga tidak dapat dispesifikasikan pada nilai batas partisi, karena juga dianggap sebagai nilai NULL.

  Supaya nilai NULL dapat masuk ke partisi, maka oracle mengurutkan nilai NULL pada nilai tertinggi daripada nilai yang lain tetapi dibawah nilai MAXVALUE.

  Artinya, anda harus membuat batas partisi teratas dengan MAXVALUE supaya nilai NULL dapat masuk ke partisi, bila tidak, maka baris data yang mengandung kunci partisi NULL akan gagal masuk ke tabel.

Batas Partisi untuk Range Partitioning

- Tipe data DATE

  jika kunci partisi mengandung kolom bertipe data DATE, maka anda harus menspesifikasikan nilai batasnya dengan fungsi TO_DATE dengan 4 karakter tahun, bilamana tidak, maka anda akan gagal membuat tabel atau index. Contoh :

```
CREATE TABLE sales_range
(salesman_id NUMBER(5),
salesman_name VARCHAR2(30),
sales_amount NUMBER(10),
sales_date DATE)
COMPRESS
PARTITION BY RANGE(sales_date)
(PARTITION sales_jan2000 VALUES LESS THAN(TO_DATE('02/01/2000','DD/MM/YYYY')),
PARTITION sales_feb2000 VALUES LESS THAN(TO_DATE('03/01/2000','DD/MM/YYYY')),
PARTITION sales_mar2000 VALUES LESS THAN(TO_DATE('04/01/2000','DD/MM/YYYY')),
PARTITION sales_apr2000 VALUES LESS THAN(TO_DATE('05/01/2000','DD/MM/YYYY')));
```

Batas Partisi untuk Range Partitioning

- Ketika anda meng-query atau memodifikasi data, direkomendasikan menggunakan fungsi TO_DATE dalam klausa where, optimizer akan lebih powerfull bila mana anda menggunakannya, sebagai contoh :

```
SELECT * FROM sales_range
WHERE sales_date BETWEEN TO_DATE('01-JUL-00', 'DD-MON-YY')
AND TO_DATE('01-OCT-00', 'DD-MON-YY');
```

Index Partisi

- Aturan index pada partisi, sama dengan index pada tabel.
- Index dapat dipartisi kecuali :
  1. Index bertipe cluster index
  2. Index didefinisikan pada tabel cluster.
- Anda dapat menggabungkan partisi dan nonpartisi index dengan partisi dan nonpartisi tabel
  1. Partisi tabel dapat mempunyai partisi partisi atau non partisi index.
  2. Nonpartisi tabel dapat mempunyai partisi atau non partisi B-Tree Index.

Index Partisi

- Index bertipe Bitmap dalam nonpartisi tabel tidak dapat dipartisi.
- Bitmap Index dalam partisi tabel harus bertipe local index.
- Namun, partisi index lebih kompleks daripada partisi tabel karena mempunyai 3 tipe partisi index

1. Local prefixed
2. Local nonprefixed
3. Global prefixed



Index Partisi

Local Partitioning Index.

- Semua kunci dalam partisi index tertentu mengacu hanya satu tabel saja.
- Keuntungan :

1. Hanya satu index partisi yang dibutuhkan pada saat dibangun kembali.
2. Durasi dari operasi maintenance partisi seimbang dengan ukuran partisi jika partisi tabel hanya mempunyai local index.
3. Local indexes mendukung partisi secara independent.



Index Partisi

4. Local indexes mendukung smooth roll-out dari data lama dan roll-in dari data baru new data dalam tabel historikal.
5. Query access plans lebih baik.
6. Local indexes menyederhanakan tugas recovery tablespace yang tidak komplit.



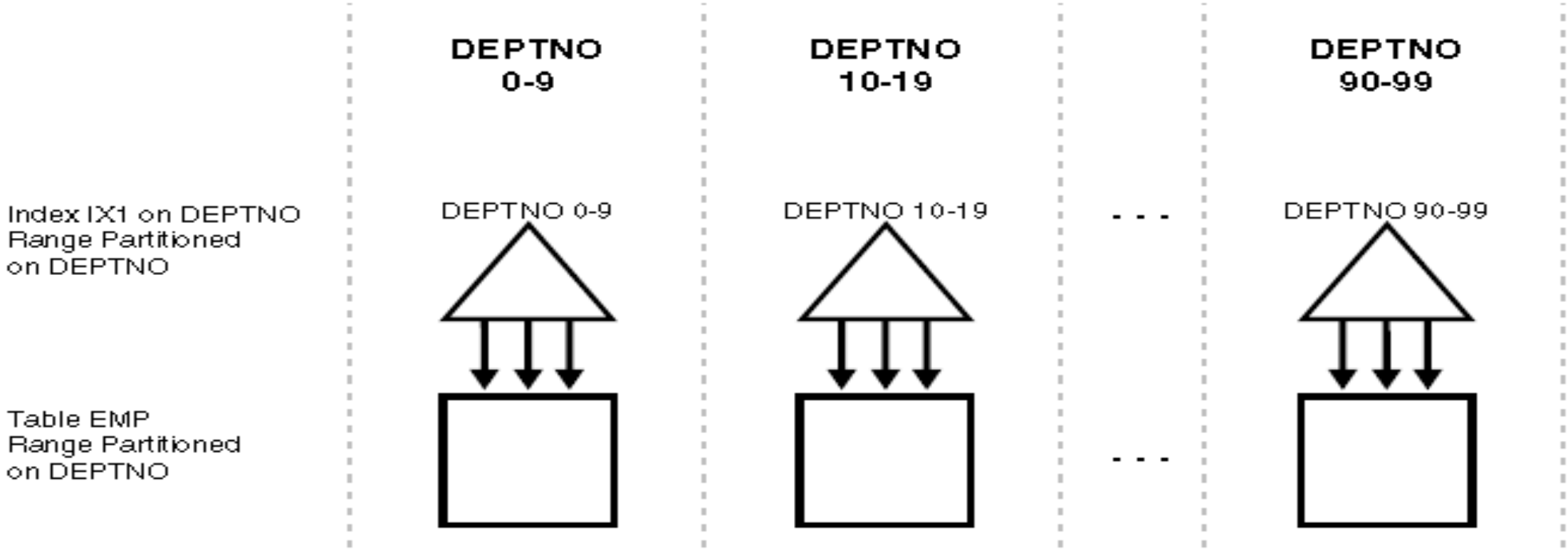
Index Partisi

1. Local prefixed index
   - Bilamana kunci partisi kolom ditempatkan disebelah kiri index kolom.
2. Local Nonprefixed index
   - Bilamana kunci partisi kolom tidak ditempatkan disebelah kiri index kolom.



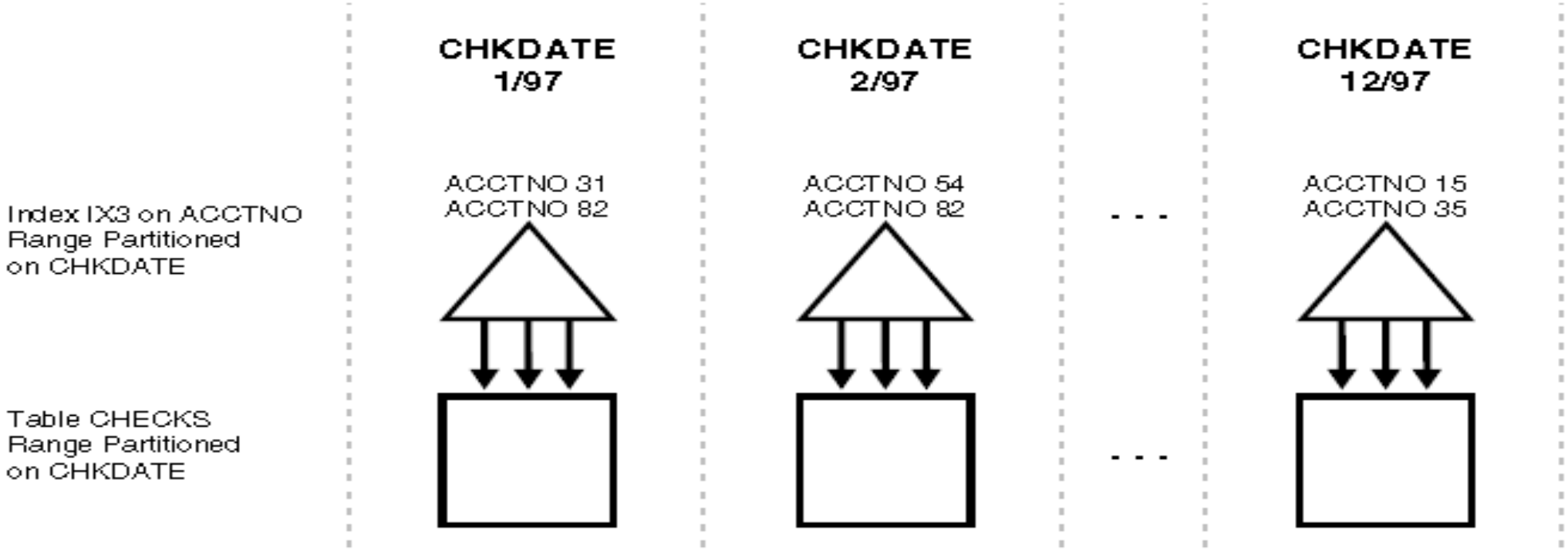
Index Partisi

Figure 5–4 Local Prefixed Index



Index Partisi

Figure 5–5 Local Nonprefixed Index



Index Partisi

- Global Partitioning Index.

  Semua kunci dalam partisi index tertentu mengacu pada dua atau lebih partisi tabel.

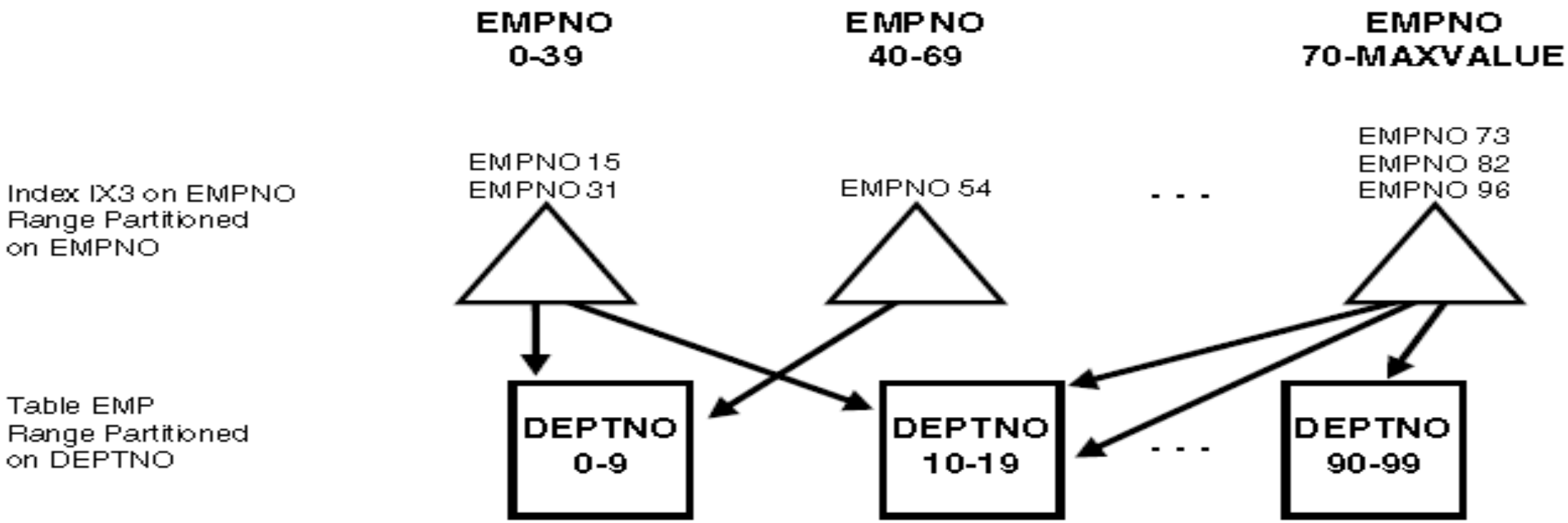
  Hanya ada Global prefixed index

  Bilamana index kolom terdapat dalam partisi kolom



Index Partisi

Figure 5–6 Global Prefixed Partitioned Index



Ringkasan Index Partisi

Table 5–2 Types of Partitioned Indexes

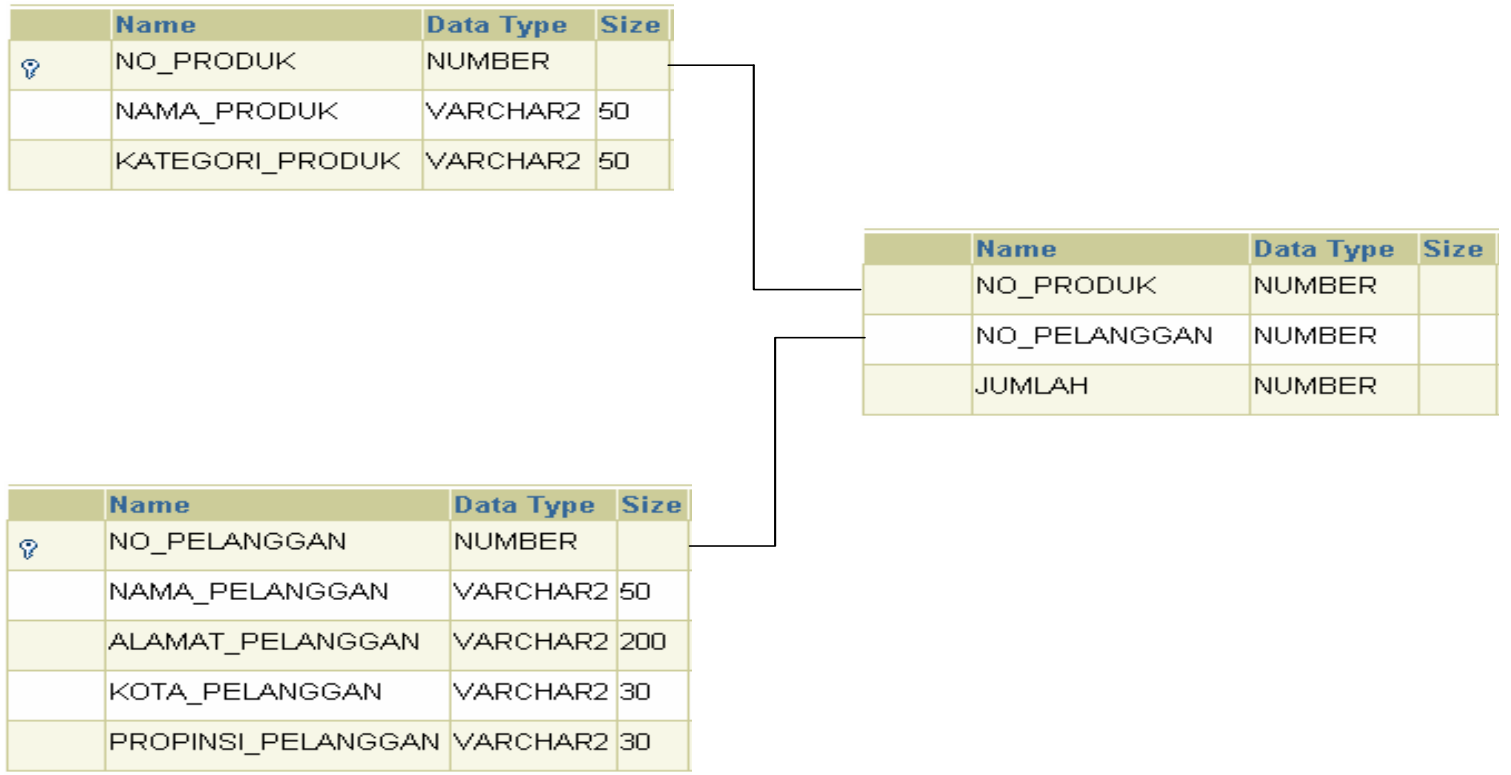
Type of Index	Index Equipartitioned with Table	Index Partitioned on Left Prefix of Index Columns	UNIQUE Attribute Allowed	Example: Table Partitioning Key	Example: Index Columns	Example: Index Partitioning Key
Local Prefixed (any partitioning method)	Yes	Yes	Yes	A	A, B	A
Local Nonprefixed (any partitioning method)	Yes	No	Yes[1]	A	B, A	A
Global Prefixed (range partitioning only)	No[2]	Yes	Yes	A	B	B

Perbandingan Index Partisi

Table 5–3 Comparing Prefixed Local, Nonprefixed Local, and Global Indexes

Index Characteristics	Prefixed Local	Nonprefixed Local	Global
Unique possible?	Yes	Yes	Yes. Must be global if using indexes on columns other than the partitioning columns
Manageability	Easy to manage	Easy to manage	Harder to manage
OLTP	Good	Bad	Good
Long Running (DSS)	Good	Good	Not Good

TUGAS



	Name	Data Type	Size
🔑	NO_PRODUK	NUMBER	
	NAMA_PRODUK	VARCHAR2	50
	KATEGORI_PRODUK	VARCHAR2	50

	Name	Data Type	Size
	NO_PRODUK	NUMBER	
	NO_PELANGGAN	NUMBER	
	JUMLAH	NUMBER	

	Name	Data Type	Size
🔑	NO_PELANGGAN	NUMBER	
	NAMA_PELANGGAN	VARCHAR2	50
	ALAMAT_PELANGGAN	VARCHAR2	200
	KOTA_PELANGGAN	VARCHAR2	30
	PROPINSI_PELANGGAN	VARCHAR2	30

- Dari desain DW diatas, buat partisi tabelnya,
  Dan Capture hasilnya, dan dikumpulkan

Referensi

- Oracle Data Warehouse and Oracle Business Intelligence Beans, Oracle Corporation, 2002.
- Lilian Hobbs, Susan Hillson and Shilpa Lawande, Oracle9iR2 Data Warehousing, Digital Press, 2003.
- Rully S, Rully Agus, InHouse Training Data Warehouse, PENS-ITS, 2005.
- Arif Djunaidy, InHouse Training Data Warehouse, PENS-ITS, 2005.
- Contour ActiveCubeX, tools for DataWarehouse

bridge to the future

http://www.eepis-its.edu